COM747 - Data Science and Machine Learning – CourseWork2 Code File

Topic: Can Machine learning be trained to predict Asthma based on symptoms?

The following contains code files for our group work, as well as our individual contributions to the success of the group work:

Habeebllahi Ejio

(Habeeb visualisation in particular boxplot)

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Read the dataset
df = pd.read_csv('Transformed_Asthma_Dataset.csv')
# Define combinations to plot
combinations = [
# Respiratory function by diagnosis
{'numeric': 'LungFunctionFEV1', 'categorical': 'Diagnosis'},
{'numeric': 'LungFunctionFVC', 'categorical': 'Diagnosis'},
# Physical characteristics by diagnosis
{'numeric': 'BMI', 'categorical': 'Diagnosis'},
{'numeric': 'Age', 'categorical': 'Diagnosis'},
{'numeric': 'PhysicalActivity', 'categorical': 'Diagnosis'},
# Environmental exposures by diagnosis
{'numeric': 'PollutionExposure', 'categorical': 'Diagnosis'},
{'numeric': 'PollenExposure', 'categorical': 'Diagnosis'},
{'numeric': 'DustExposure', 'categorical': 'Diagnosis'},
# Lifestyle factors by diagnosis
{'numeric': 'DietQuality', 'categorical': 'Diagnosis'},
{'numeric': 'SleepQuality', 'categorical': 'Diagnosis'},
# Respiratory function by symptoms
{'numeric': 'LungFunctionFEV1', 'categorical': 'Wheezing'},
{'numeric': 'LungFunctionFEV1', 'categorical': 'ShortnessOfBreath'},
{'numeric': 'LungFunctionFEV1', 'categorical': 'ChestTightness'},
{'numeric': 'LungFunctionFEV1', 'categorical': 'NighttimeSymptoms'},
{'numeric': 'LungFunctionFEV1', 'categorical': 'ExerciseInduced'},
# Respiratory function by risk factors
{'numeric': 'LungFunctionFEV1', 'categorical': 'Smoking'},
{'numeric': 'LungFunctionFEV1', 'categorical': 'BMI_Type'},
{'numeric': 'LungFunctionFEV1', 'categorical': 'FamilyHistoryAsthma'},
{'numeric': 'LungFunctionFEV1', 'categorical': 'HistoryOfAllergies'},
# BMI relationships
```

```
{'numeric': 'BMI', 'categorical': 'Gender'},
{'numeric': 'BMI', 'categorical': 'Ethnicity'},
{'numeric': 'BMI', 'categorical': 'Smoking'},
# Age relationships
{'numeric': 'Age', 'categorical': 'Smoking'},
{'numeric': 'Age', 'categorical': 'HistoryOfAllergies'},
{'numeric': 'Age', 'categorical': 'ExerciseInduced'},
# Physical activity relationships
{'numeric': 'PhysicalActivity', 'categorical': 'Gender'},
{'numeric': 'PhysicalActivity', 'categorical': 'BMI_Type'},
{'numeric': 'PhysicalActivity', 'categorical': 'ExerciseInduced'},
# Environmental exposures by symptoms
{'numeric': 'PollenExposure', 'categorical': 'HayFever'},
{'numeric': 'DustExposure', 'categorical': 'Wheezing'},
{'numeric': 'PollutionExposure', 'categorical': 'ShortnessOfBreath'}
]
# Create a figure for each combination
for combo in combinations:
numeric_var = combo['numeric']
categorical_var = combo['categorical']
plt.figure(figsize=(10, 6))
# Create the boxplot
sns.boxplot(x=categorical_var, y=numeric_var, data=df)
# Add title and labels
plt.title(f'Distribution of {numeric_var} by {categorical_var}', fontsize=16)
plt.xlabel(categorical_var, fontsize=14)
plt.ylabel(numeric_var, fontsize=14)
# Add a grid for easier reading
plt.grid(axis='y', linestyle='--', alpha=0.7)
# Save the figure
plt.savefig(f'boxplot_{numeric_var}_by_{categorical_var}.png', dpi=300,
bbox_inches='tight')
# Show the plot
plt.show()
```

Naren Kumar Narayanan
(Naren did share code that also helped with quite a number of visualisations)

```
# Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
# Set the style for plots
plt.style.use('seaborn-v0_8-whitegrid')
sns.set(font_scale=1.2)
# Display settings for better visualization
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
# Load the data
df = pd.read_excel('/Users/narenkumar/Desktop/data science amanda/Data.xlsx')
# Display basic information
print("Dataset shape:", df.shape)
print("\nFirst few rows:")
display(df.head())
# Check for missing values
print("\nMissing values per column:")
print(df.isnull().sum())
# Display summary statistics
print("\nSummary statistics:")
display(df.describe())
```

1. Data Loading and Exploration. First, let's load the dataset and explore its structure.

```
# Load the data
df = pd.read_excel('/Users/narenkumar/Desktop/data science amanda/Data.xlsx')
# Display basic information
print("Dataset shape:", df.shape)
print("\nFirst few rows:")
display(df.head())
# Check for missing values
print("\nMissing values per column:")
print(df.isnull().sum())
```

```
# Display summary statistics
print("\nSummary statistics:")
display(df.describe())
```

2. Data Preprocessing. Let's preprocess the data by converting categorical variables to more meaningful representations.

```
# Create a copy of the dataframe for analysis
df_analysis = df.copy()
# Convert categorical variables
# Assuming Gender: 0=Male, 1=Female
df_analysis['Gender'] = df_analysis['Gender'].map({0: 'Male', 1: 'Female'})
# Assuming Diagnosis: 0=No Asthma, 1=Asthma
df_analysis['Diagnosis'] = df_analysis['Diagnosis'].map({0: 'No Asthma', 1: 'Asthma'})
# Display the updated dataframe
display(df_analysis.head())
```

3. Correlation Analysis. Let's examine the correlations between numerical variables in the dataset.

```
# Create a correlation matrix for numerical variables
numeric_df = df.select_dtypes(include=[np.number])
correlation_matrix = numeric_df.corr()
# Plot correlation heatmap
plt.figure(figsize=(14, 12))
sns.heatmap(correlation_matrix, annot=False, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Matrix of Numerical Variables')
plt.tight_layout()
plt.show()
# Display the correlation with Diagnosis
diagnosis_corr = correlation_matrix['Diagnosis'].sort_values(ascending=False)
print("\nCorrelation with Diagnosis:")
print(diagnosis_corr)
```

4. Symptom Analysis by Diagnosis. Let's analyse the relationship between symptoms and diagnosis, including p-values from
chi-square tests.

```
# Display the pre-generated image
from IPython.display import Image
Image(filename='plots/symptoms_by_diagnosis.png')
# Analyze relationship between symptoms and diagnosis with p-values
symptom_cols = ['Wheezing', 'ShortnessOfBreath', 'ChestTightness', 'Coughing',
'NighttimeSymptoms', 'ExerciseInduced']
# Calculate chi-square p-values for each symptom
symptom_pvalues = {}
for symptom in symptom_cols:
contingency_table = pd.crosstab(df[symptom], df['Diagnosis'])
```

```
chi2, p, dof, expected = stats.chi2_contingency(contingency_table)
symptom_pvalues[symptom] = p
# Display p-values
print("Chi-square test p-values for symptoms vs diagnosis:")
for symptom, p_value in symptom_pvalues.items():
significance = "Significant" if p_value < 0.05 else "Not significant"
print(f"{symptom}: p-value = {p_value:.4f} ({significance})")
```

5. Age and BMI Analysis. Let's examine the relationship between age, BMI, and diagnosis.

```
# Display the pre-generated images
from IPython.display import Image, display
print("Age Distribution by Diagnosis:")
display(Image(filename='plots/age_distribution.png'))
print("\nBMI Distribution by Diagnosis:")
display(Image(filename='plots/bmi_boxplot.png'))
# T-test for BMI between diagnosis groups
asthma_bmi = df[df['Diagnosis'] == 1]['BMI']
no_asthma_bmi = df[df['Diagnosis'] == 0]['BMI']
t_stat, p_value = stats.ttest_ind(asthma_bmi, no_asthma_bmi, equal_var=False)
print(f"T-test for BMI between Asthma and No Asthma groups:")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Significance: {'Significant' if p_value < 0.05 else 'Not significant'}")
# T-test for Age between diagnosis groups
asthma_age = df[df['Diagnosis'] == 1]['Age']
no_asthma_age = df[df['Diagnosis'] == 0]['Age']
t_stat, p_value = stats.ttest_ind(asthma_age, no_asthma_age, equal_var=False)
print(f"\nT-test for Age between Asthma and No Asthma groups:")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Significance: {'Significant' if p_value < 0.05 else 'Not significant'}")
```

6. Lung Function Analysis. Let's analyse lung function metrics (FEV1 and FVC) by diagnosis.

```
# Display the pre-generated image
Image(filename='plots/lung_function.png')
# T-test for lung function between diagnosis groups
asthma_fev1 = df[df['Diagnosis'] == 1]['LungFunctionFEV1']
no_asthma_fev1 = df[df['Diagnosis'] == 0]['LungFunctionFEV1']
t_stat, p_value = stats.ttest_ind(asthma_fev1, no_asthma_fev1, equal_var=False)
print(f"T-test for FEV1 between Asthma and No Asthma groups:")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Significance: {'Significant' if p_value < 0.05 else 'Not significant'}")
```

```
asthma_fvc = df[df['Diagnosis'] == 1]['LungFunctionFVC']
no_asthma_fvc = df[df['Diagnosis'] == 0]['LungFunctionFVC']
t_stat, p_value = stats.ttest_ind(asthma_fvc, no_asthma_fvc, equal_var=False)
print(f"\nT-test for FVC between Asthma and No Asthma groups:")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Significance: {'Significant' if p_value < 0.05 else 'Not significant'}")
```

7. Risk Factors Analysis. Let's analyse various risk factors and their relationship with diagnosis.

```
# Display the pre-generated image
Image(filename='plots/risk_factors.png')
# Risk factors analysis with p-values
risk_factors = ['Smoking', 'PhysicalActivity', 'PollutionExposure',
'PollenExposure', 'DustExposure', 'PetAllergy',
'FamilyHistoryAsthma', 'HistoryOfAllergies', 'Eczema', 'HayFever']
# Calculate p-values for each risk factor
risk_factor_pvalues = {}
for factor in risk_factors:
if df[factor].nunique() <= 2:
# For binary factors, use chi-square test
contingency_table = pd.crosstab(df[factor], df['Diagnosis'])
chi2, p, dof, expected = stats.chi2_contingency(contingency_table)
test_type = "Chi-square"
else:
# For continuous factors, use t-test
asthma_val = df[df['Diagnosis'] == 1][factor]
no_asthma_val = df[df['Diagnosis'] == 0][factor]
t_stat, p = stats.ttest_ind(asthma_val, no_asthma_val, equal_var=False)
test_type = "T-test"
risk_factor_pvalues[factor] = (p, test_type)
# Display p-values
print("Statistical test p-values for risk factors vs diagnosis:")
for factor, (p_value, test_type) in risk_factor_pvalues.items():
significance = "Significant" if p_value < 0.05 else "Not significant"
print(f"{factor}: p-value = {p_value:.4f} ({test_type}, {significance})")
```

8. Logistic Regression Analysis. Let's perform logistic regression to identify significant predictors of diagnosis.

```
# Display the pre-generated image
Image(filename='plots/logistic_regression_coefs.png')
# Prepare data for logistic regression
X = df[['Age', 'BMI', 'Smoking', 'PhysicalActivity', 'PollutionExposure',
'PollenExposure', 'DustExposure', 'PetAllergy', 'FamilyHistoryAsthma',
```

```
'HistoryOfAllergies', 'Eczema', 'HayFever', 'LungFunctionFEV1',
'LungFunctionFVC', 'Wheezing', 'ShortnessOfBreath', 'ChestTightness',
'Coughing', 'NighttimeSymptoms', 'ExerciseInduced']]
y = df['Diagnosis']
# Fit statsmodels logistic regression for p-values
X_sm = sm.add_constant(X)
logit_model = sm.Logit(y, X_sm)
result = logit_model.fit()
# Display summary
print("Logistic Regression Summary:")
print(result.summary())
# Extract p-values and create a summary table
pvalues = result.pvalues[1:]
coefs = result.params[1:]
odds_ratios = np.exp(coefs)
# Create a summary dataframe
summary_df = pd.DataFrame({
'Variable': X.columns,
'Coefficient': coefs,
'Odds Ratio': odds_ratios,
'p-value': pvalues,
'Significant': pvalues < 0.05
})
# Sort by p-value
summary_df = summary_df.sort_values('p-value')
# Display the summary
display(summary_df)
```

Amanda Logan McCook – R studio

```
#removing columns
cols_to_drop<-c("DoctorInCharge", "GastroesophagealReflux","EducationLevel")
for (c in cols_to_drop){
  asthma_disease_data[c]<-NULL
}
#Removed unneeded columns from the dataset
#adding column for BMI - classifying data
classify_weight <- function(BMI){#function on the BMI and defining
  if (BMI <18.5){
    return("Underweight")
  }
  else if (BMI <=24.9){
    return("Healthy Weight")
  }
  else if (BMI <= 29.9){
    return("Overweight")
  }
  else if(BMI <=39.9){
    return("Obese")
  }
  else {
    return ("Severely Obese")
  }
}
asthma_disease_data$BMI_Type <- sapply(asthma_disease_data$BMI, classify_weight)
install.packages("writexl")
library("writexl")
#This code exported the data into an xml formal which I then will save as CSV for python
coding elements.
write_xlsx(asthma_disease_data,"C:/Users/iamab/OneDrive/Desktop/AI
UUJ/2025/Assignment/Data_Clean25.xlsx")
```

Amanda Logan McCook's Python File
(The core code contains the training and testing using Random Forest and Decision Tree, and contains Feature Engineering as well as Confusion Matrix)

```
#Libaries that I will need for code
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split#Training the test split - will be defined
later
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.tree import DecisionTreeClassifier, plot_tree
```

```
#Load Dataset

def load_data(file_path):
    df = pd.read_csv(file_path)
    return df
#Parameters: Load the dataset from the given file path.Parameters: file_path (str): Path to the
CSV file containing the asthma dataset.
#Returns:DataFrame: A pandas DataFrame containing the dataset.
#Data Cleaning - Cleaning the dataset by removing duplicate entries and unnecessary
columns.
#The 'PatientID' column is removed as it does not contribute to predictive modelling.

def clean_data(df):
    df = df.drop_duplicates() #Remving duplicates
    df = df.drop(columns=["PatientID"])  #Ensuring 'PatientID' does not hold useful predictive
patterns before dropping.
    return df
#Exploratory Data Analysis
#Perform basic exploratory data analysis (EDA) to understand the dataset.
#This includes displaying summary statistics, checking for missing values, and visualising
key data distributions.
#Parameters: df (DataFrame): The cleaned dataset.

def explore_data(df):
    print("Data Summary:")
    print(df.describe())
    print("\nMissing Values:")
```

```
    print(df.isnull().sum())

#Visualisations
    plt.figure(figsize=(10, 5))
    sns.countplot(x='Diagnosis', data=df)
    plt.title("Asthma Diagnosis Distribution")
    plt.show()

    plt.figure(figsize=(10, 5))
    sns.boxplot(x='Diagnosis', y='Age', data=df)
    plt.title("Age Distribution by Diagnosis")
    plt.show()
#Feature Engineering

def preprocess_data(df):
    if 'Diagnosis' in df.columns:
        if df['Diagnosis'].dtype == 'object':  #If Diagnosis is categorical (e.g., 'Yes', 'No')
            le = LabelEncoder()  #Use LabelEncoder to convert to 0 and 1
            df['Diagnosis'] = le.fit_transform(df['Diagnosis'])
        elif df['Diagnosis'].dtype in ['float64', 'int64']:
#If it's continuous, binarize it
            threshold = df['Diagnosis'].median()
            df['Diagnosis'] = (df['Diagnosis'] > threshold).astype(int)

#Debugging: Confirm Diagnosis transformation is successful
        print("Unique values in 'Diagnosis' after preprocessing:", df['Diagnosis'].unique())
        print("Data type of 'Diagnosis':", df['Diagnosis'].dtype)

#Encode categorical variables and scale numerical features to prepare data for modelling.
#Categorical variables are converted to numerical format using Label Encoding, and
numerical features are standardised using StandardScaler.
#Parameters:df (DataFrame): The dataset after cleaning. Returns:DataFrame: The dataset
with encoded categorical variables and scaled numerical features.

    le = LabelEncoder()#Defining number for the label - will display numerically
    categorical_cols = df.select_dtypes(include=['object']).columns
    for col in categorical_cols:
        df[col] = le.fit_transform(df[col])

#Scale numerical features
    scaler = StandardScaler()
    numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns
```

```
    df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

    return df
#Train Model - Train a machine learning model using Random Forest to predict asthma
diagnosis.
#The dataset is split into training and test sets, and the model is evaluated using
#Accuracy, classification report, and a confusion matrix. Parameters: df (DataFrame): The
pre-processed dataset.

#Train Model
def train_model(df):
    X = df.drop(columns=['Diagnosis'])  #Features
    y = df['Diagnosis']  #Target variable

#Debugging: Ensure y is properly formatted
    print("\nDebugging: Target variable (y) properties")
    print("Unique values in y:", y.unique())
    print("Data type of y:", y.dtype)

#Explicitly convert y to a NumPy array of integer type
    y = y.values.astype(int)  # Ensure y is an integer array
    print("\nAfter explicit conversion:")
    print("Unique values in y:", np.unique(y))
    print("Data type of y:", type(y), "with dtype:", y.dtype)

#Split data into training/testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialise and train the Random Forest Classifier
    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)

#Make predictions
    y_pred = model.predict(X_test)

#Evaluate the model
    print("\nModel Performance:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Classification Report:")
    print(classification_report(y_test, y_pred, zero_division=0))

#Confusion Matrix
```

```
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title("Confusion Matrix")
    plt.show()
def train_DT_model(df):
    X = df.drop(columns=['Diagnosis'])  #Features
    y = df['Diagnosis']  #Target variable

#Debugging: Ensure y is properly formatted
    print("\nDebugging: Target variable (y) properties")
    print("Unique values in y:", y.unique())
    print("Data type of y:", y.dtype)

#Explicitly convert y to a NumPy array of integer type
    y = y.values.astype(int)  # Ensure y is an integer array
    print("\nAfter explicit conversion:")
    print("Unique values in y:", np.unique(y))
    print("Data type of y:", type(y), "with dtype:", y.dtype)

#Split data into training/testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

#Initialise and train the Random Forest Classifier
    model = DecisionTreeClassifier(class_weight='balanced', max_depth=4, random_state=42)
    model.fit(X_train, y_train)

#Make predictions
    y_pred = model.predict(X_test)

#Evaluate the model
    print("\nModel Performance:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Classification Report:")
    print(classification_report(y_test, y_pred, zero_division=0))

#Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
```

```
    plt.title("Confusion Matrix")
    plt.show()

#Plot the Decision Tree
    plt.figure(figsize=(20, 10))  #Adjust depending on the tree's size
    plot_tree(
        model,
        # feature_names=feature_names,
        # class_names=class_names,
        filled=True,
        rounded=True
        )
    plt.title("Decision Tree Visualization")
    plt.show()

    return model
#Training a DecisionTreeClassifier and visualising the resulting decision tree.
def train_and_plot_decision_tree(X, y, feature_names, class_names):

#Ensure the target variable is properly formatted
    print("\nDebugging: Target variable (y)")
    print("Unique values in y:", np.unique(y))
    print("Data type of y:", type(y), "with dtype:", y.dtype)

#Explicitly convert y to integer type if necessary
    if not np.issubdtype(y.dtype, np.integer):
        print("Warning: Target variable is not discrete! Converting to integer...")
        y = y.astype(int)
#File path defined
file_path = "Documents/Transformed_Asthma_Dataset (1).csv"
df = load_data(file_path)
#Cleaning the data
df_Clean = clean_data(df)
#Exploring the data that is clean
explore_data(df_Clean)
#Preprocess data in the clean 0 and 1 to represent diagnosis
df_Pre = preprocess_data(df_Clean)
df_Pre.head()#Shows the table and values
train_model(df_Pre) #Random Forest
train_DT_model(df_Pre) #Decision tree
#Main Execution
if __name__ == "__main__":
```

```
    file_path = "Documents/Transformed_Asthma_Dataset (1).csv"
    df = load_data(file_path)
    df = clean_data(df)
    explore_data(df)
    df = preprocess_data(df)
    train_model(df)

#Prepare features and target
    X = df.drop(columns=['Diagnosis'])  # Features
    y = df['Diagnosis']  # Target variable

#Double-check the target variable
    print("Target (Diagnosis) distribution:\n", y.value_counts())
    print("Target data type:", y.dtype)

#Feature and class names
    feature_names = X.columns.tolist()
    class_names = ['No', 'Yes']  # Adapt if necessary

#Train and visualize the decision tree
    train_and_plot_decision_tree(X, y, feature_names, class_names)

#Ethical Considerations and Sustainability
# - Ensure patient data is anonymised and used ethically.
# - Avoid biases in the dataset, such as underrepresenting certain ethnicities or genders.
# - Model fairness should be evaluated to prevent discrimination in asthma diagnosis.
# - Consider the environmental impact of computational resources used in model training.
```

Aprem George
(The core of this code helps to ascertain how much we can predict asthma based on the feature/symptoms that were mentioned in the original Dataset along with clustering)

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker
df = pd.read_csv("Transformed_Asthma_Dataset.csv")
print(df.head())  # View first few rows
print(df.info())  # Check data types
print(df.describe())  # Get summary statistics
category_col = "Diagnosis"
# possible cause factors of asthma (having numerical values)
numerical_cols = ["BMI","PhysicalActivity", "DietQuality","SleepQuality",
"PollutionExposure", "PollenExposure", "DustExposure"]
df_melted = df.melt(id_vars=[category_col],
                    value_vars=numerical_cols,
                    var_name="Metric",
                    value_name="Value")
plt.figure(figsize=(30, 18))  # Increased figure size
# create boxplot of cause factors by diagnosis result
sns.boxplot(x=category_col, y="Value", hue="Metric", data=df_melted, palette="Set2")

plt.title("Comparison of possible contributors(having numerical values) by diagnosis result",
fontsize=16)
plt.xlabel("Diagnosis (Yes/No)", fontsize=14)
plt.ylabel("Value", fontsize=14)

# Set y-axis scale with a step of 1
y_min = int(df_melted["Value"].min())  # Convert to integer to avoid float-based ticks
y_max = int(df_melted["Value"].max()) + 1
plt.ylim(y_min, y_max)
plt.yticks(range(y_min, y_max + 1, 1))  # Step of 1

plt.legend(title="Metric", bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0., fontsize=20,
markerscale=3)
plt.grid(True)
plt.show()
plt.savefig("boxplot.png", dpi=300, bbox_inches="tight")
plt.figure(figsize=(30, 18))  # Increased figure size
```

```
# Create a point plot instead of a box plot
sns.pointplot(x=category_col, y="Value", hue="Metric", data=df_melted, palette="Set2",
              dodge=True, markers="o", linestyles="--",
              markersize=10, linewidth=3)  # Using markersize and linewidth instead of scale

plt.title("Comparison of possible contributors (having numerical values) by diagnosis result",
fontsize=16)
plt.xlabel("Diagnosis (Yes/No)", fontsize=14)
plt.ylabel("Value", fontsize=14)

# Set y-axis scale with a step of 1
y_min = int(df_melted["Value"].min())  # Convert to integer to avoid float-based ticks
y_max = int(df_melted["Value"].max()) + 1
plt.ylim(y_min, y_max)
plt.yticks(range(y_min, y_max + 1, 1))  # Step of 1

# Adjust legend position and font size
plt.legend(title="Metric", bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0., fontsize=20,
markerscale=3)

plt.grid(True)
plt.show()
numerical_cols = ["LungFunctionFEV1", "LungFunctionFVC"]
df_melted = df.melt(id_vars=[category_col],
                    value_vars=numerical_cols,
                    var_name="Metric",
                    value_name="Value")
plt.figure(figsize=(30, 18))  # Increased figure size
sns.boxplot(x=category_col, y="Value", hue="Metric", data=df_melted, palette="Set2")

plt.title("Comparison of lung health measurements by diagnosis result", fontsize=24)
plt.xlabel("Diagnosis (Yes/No)", fontsize=22)
plt.ylabel("Value", fontsize=22)

# Set y-axis scale with a step of 1
y_min = int(df_melted["Value"].min())  # Convert to integer to avoid float-based ticks
y_max = int(df_melted["Value"].max()) + 1
plt.ylim(y_min, y_max)
plt.yticks(np.arange(np.floor(y_min), np.ceil(y_max), 0.5))

plt.legend(title="Metric", bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0., fontsize=24,
markerscale=4)
```

```
# Increase size of x and y axis tick labels
plt.xticks(fontsize=20)  # Larger x-axis ticks (Yes/No)
plt.yticks(fontsize=20)  # Larger y-axis ticks (numbers)
plt.grid(True)
plt.show()
# Convert categorical Yes/No values into numerical 1/0. This is done for createing a heatmap,
since heatmap can only be applied on numerical columns
df_binary = df.replace({"Yes": 1, "No": 0})

# Display the transformed dataset
print(df_binary.head())
# Exclude columns which are not symptoms
df_symptoms = df_binary.drop(columns=["PatientID",
"Age","Gender","Ethnicity","BMI","BMI_Type","Smoking","PhysicalActivity","DietQuality
","SleepQuality","PollutionExposure","PollenExposure","DustExposure","PetAllergy","Fami
lyHistoryAsthma","HistoryOfAllergies","LungFunctionFEV1","LungFunctionFVC","Exercis
eInduced"])

plt.figure(figsize=(10, 6))  # Adjust figure size

# Generate heatmap of correlations between possible symptoms and diagnosis
sns.heatmap(df_symptoms.corr(), annot=True, cmap="coolwarm", cbar=True,
linewidths=0.5)

plt.title("Heatmap of Correlations Between Symptoms and Diagnosis")
plt.show()
df_symptoms = df_binary.drop(columns=["PatientID",
"Age","Gender","Ethnicity","BMI","BMI_Type","Smoking","PhysicalActivity","DietQuality
","SleepQuality","PollutionExposure","PollenExposure","DustExposure","PetAllergy","Fami
lyHistoryAsthma","HistoryOfAllergies","LungFunctionFEV1","LungFunctionFVC","Exercis
eInduced"])  # Exclude non-numeric columns

plt.figure(figsize=(10, 6))  # Adjust figure size
sns.heatmap(df_symptoms.groupby("Diagnosis").mean(), annot=True, cmap="coolwarm",
cbar=True)

plt.title("Heatmap of Average Yes/No Occurrences of symptoms by Diagnosis")
plt.show()
# In below cells we are finding the mean and standard deviation of possible cause factors(
having numerical values) in order to estimate a benchmark value for that particular cause
factor.
```

```
# By using this benchmark value as a criteria the column values can be converted to either 0 or 1 which is suitable for heatmap generation.
column_name = "DietQuality"

mean_value = df[column_name].mean()  # DietQuality mean
std_dev_value = df[column_name].std()  # DietQuality standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")

column_name = "SleepQuality"

mean_value = df[column_name].mean()  # SleepQuality mean
std_dev_value = df[column_name].std()  # SleepQuality standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")
column_name = "PollutionExposure"

mean_value = df[column_name].mean()  # PollutionExposure mean
std_dev_value = df[column_name].std()  # PollutionExposure standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")
column_name = "PollenExposure"

mean_value = df[column_name].mean()  # PollenExposure mean
std_dev_value = df[column_name].std()  # PollenExposure standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")
column_name = "DustExposure"

mean_value = df[column_name].mean()  # DustExposure mean
std_dev_value = df[column_name].std()  # DustExposure standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")
# The column values are converted to either 0 or 1 based on the respective benchmark values.

# Change it to 1 if less than the benchmark value (factors which should be flagged if less than the benchmark value). Else change it to zero.
```

```
df["PhysicalActivity"] = np.where(df["PhysicalActivity"] < 3.5, 1, 0)

df["DietQuality"] = np.where(df["DietQuality"] < 3.6, 1, 0)

df["SleepQuality"] = np.where(df["SleepQuality"] < 5, 1, 0)

# Change it to 1 if greater than the benchmark value (factors which should be flagged if
greater than the benchmark value)

df["PollutionExposure"] = np.where(df["PollutionExposure"] > 7.93, 1, 0)

df["PollenExposure"] = np.where(df["PollenExposure"] > 8, 1, 0)

df["DustExposure"] = np.where(df["DustExposure"] > 7.83, 1, 0)

print(df.head())
# The column values are converted to either 0 or 1 based on the benchmark values of BMI.

df["BMI"] = np.where((df["BMI"] < 16) | (df["BMI"] > 30), 1, 0)
print(df.head(20))
# Convert categorical Yes/No values into numerical 1/0. This is done for createing a heatmap,
since heatmap can only be applied on numerical columns
df_binary = df.replace({"Yes": 1, "No": 0})

# Display the transformed dataset
print(df_binary.head())
# Include only possible cause factor columns
cause_cols = ["Diagnosis", "BMI", "Smoking",
"PhysicalActivity","DietQuality","SleepQuality","PollutionExposure","PollenExposure","Du
stExposure","PetAllergy","FamilyHistoryAsthma","HistoryOfAllergies"]
df_cause = df_binary[cause_cols]

plt.figure(figsize=(10, 6))  # Adjust figure size
sns.heatmap(df_cause.groupby("Diagnosis").mean(), annot=True, cmap="coolwarm",
cbar=True)

plt.title("Heatmap of Average Yes/No Occurrences of causes by Diagnosis")
plt.show()
# Compute the FEV1/FVC Ratio
df["FEV1_FVC_Ratio"] = df["LungFunctionFEV1"] / df["LungFunctionFVC"]
# Apply Asthma Criteria
```

```
# Asthma is likely if:
# FEV1 < 7 (indicative of airflow limitation)
# FEV1/FVC Ratio < 0.7 (suggesting obstruction)

df["Possible_Asthma"] = (df["LungFunctionFEV1"] < 7) & (df["FEV1_FVC_Ratio"] < 0.7)
print(df.head())
# Convert True and False values of Possible_Asthma column into numerical 1/0.
df_binary = df.replace({True: 1, False: 0}).replace({"Yes": 1, "No": 0})
print(df_binary.head())
# Create Heatmap of asthma detection based on LungFunctionFEV1 and LungFunctionFVC.
detection_cols = ["Diagnosis", "Possible_Asthma"]
df_detection = df_binary[detection_cols]

plt.figure(figsize=(10, 6))  # Adjust figure size
sns.heatmap(df_detection.groupby("Diagnosis").mean(), annot=True, cmap="coolwarm",
cbar=True)

plt.title("Heatmap of asthma detection based on LungFunctionFEV1 and
LungFunctionFVC")
plt.show()
```

Aprem George – unsupervised learning for Asthma dataset

```
import pandas as pd
import numpy as np
df = pd.read_csv("Transformed_Asthma_Dataset.csv")
print(df.head())  # View first few rows
print(df.info())  # Check data types
print(df.describe())  # Get summary statistics
# Since machine learning models work best with numerical data, convert categorical
variables into numerical format.
# Convert Yes/No columns to 1/0
df.replace({"Yes": 1, "No": 0}, inplace=True)
print(df.head())
# For true categorical features without inherent order use One-Hot Encoding.
categorical_cols = ["Gender", "Ethnicity"]

df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=False)
print(df_encoded.head())

# K-Means relies on Euclidean distance, which needs numerical values. Hence convert any
boolean columns to integers
for col in df_encoded.columns:
    if df_encoded[col].dtype == bool:
        df_encoded[col] = df_encoded[col].astype(int)
print(df_encoded.head())
# Exclude columns which are irrelevent
df_encoded = df_encoded.drop(columns=["PatientID","BMI_Type"])
print(df_encoded.head())

#Standardize the Features
#This ensures all features have equal weight in distance calculations.

!pip install scikit-learn

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_standardized = scaler.fit_transform(df_encoded)
# After standardization the dataset is a NumPy array. And NumPy arrays do not have the
head() method that is specific to Pandas DataFrames.
# To resolve this, we are converting df_standardized back to a Pandas DataFrame

df_standardized = pd.DataFrame(df_standardized, columns=df_encoded.columns)
```

```
print(df_standardized.head())
#Use the Elbow Method to Choose k (number of clusters)

import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

inertia = []
k_range = range(1, 50)

for k in k_range:
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(df_standardized)
    inertia.append(model.inertia_)

plt.figure(figsize=(8, 5))
plt.plot(k_range, inertia, marker='o')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.title('Elbow Method For Optimal k')
plt.grid(True)
plt.show()
# Optimal K calculated from elbow method = 14 ("elbow" point where the drop in inertia
slows)
optimal_k = 14

kmeans = KMeans(n_clusters=optimal_k, random_state=42)
clusters = kmeans.fit_predict(df_standardized)

# Add cluster labels back to the original dataframe
df_encoded["Cluster"] = clusters
print(df_encoded.head())
# Analyze the Clusters
# View basic cluster distribution
print(df_encoded["Cluster"].value_counts())

# Check cluster-wise means
cluster_means = df_encoded.groupby("Cluster")["Diagnosis"].mean()
print(cluster_means)
# List of feature columns you used for clustering (excluding "Cluster")
features = [col for col in df_encoded.columns if col != "Cluster"]

# Calculate the mean of each feature per cluster
```

```
cluster_summary = df_encoded.groupby("Cluster")[features].mean().round(2)
print(cluster_summary)

# Export as Excel File
!pip install openpyxl
cluster_summary.to_excel("cluster_summary.xlsx", index=True)
import seaborn as sns

plt.figure(figsize=(20, 10))
sns.heatmap(cluster_summary, annot=True, cmap="coolwarm")
plt.title("Cluster Feature Means")
plt.tight_layout()

# Save the heatmap as a PNG image
plt.savefig("cluster_summary_heatmap.png", dpi=300)
plt.show()
```

Combined Code Working: All code works together. Amanda Logan McCook put this together as one single code document. I did have to edit some of the code to show charts because it used libraries that weren't installed.

```
#Libaries that I will need for code - Both Habeeb & Amanda used
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split#Training the test split - will be defined
later
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
from sklearn.tree import DecisionTreeClassifier, plot_tree

#Naren
from scipy import stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

# Set the style for plots -
plt.style.use('seaborn-v0_8-whitegrid')
sns.set(font_scale=1.2)
#Display settings for better visualization
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

#Aprem
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

#Start of Code - Amanda
#Load Dataset

def load_data(file_path):
    df = pd.read_csv(file_path)
    return df

#Parameters: Load the dataset from the given file path. Parameters:file_path (str): Path to the
CSV file containing the asthma dataset.
#Returns:DataFrame: A pandas DataFrame containing the dataset.

#Data Cleaning - Cleaning the dataset by removing duplicate entries and unnecessary
columns.
#The 'PatientID' column is removed as it does not contribute to predictive modelling.
```

```
def clean_data(df):
    df = df.drop_duplicates() #Remving duplicates
    df = df.drop(columns=["PatientID"])  #Ensuring 'PatientID' does not hold useful predictive
patterns before dropping.
    return df

#Exploratory Data Analysis
#Perform basic exploratory data analysis (EDA) to understand the dataset.
#This includes displaying summary statistics, checking for missing values, and visualising
key data distributions.
#Parameters: df (DataFrame): The cleaned dataset.

def explore_data(df):
    print("Data Summary:")
    print(df.describe())
    print("\nMissing Values:")
    print(df.isnull().sum())

#Visualizations
    plt.figure(figsize=(10, 5))
    sns.countplot(x='Diagnosis', data=df)
    plt.title("Asthma Diagnosis Distribution")
    plt.show()

    plt.figure(figsize=(10, 5))
    sns.boxplot(x='Diagnosis', y='Age', data=df)
    plt.title("Age Distribution by Diagnosis")
    plt.show()

#Feature Engineering

def preprocess_data(df):
    if 'Diagnosis' in df.columns:
        if df['Diagnosis'].dtype == 'object':  #If Diagnosis is categorical (e.g., 'Yes', 'No')
            le = LabelEncoder()  #Use LabelEncoder to convert to 0 and 1
            df['Diagnosis'] = le.fit_transform(df['Diagnosis'])
        elif df['Diagnosis'].dtype in ['float64', 'int64']:
#If it's continuous, binarize it
            threshold = df['Diagnosis'].median()
            df['Diagnosis'] = (df['Diagnosis'] > threshold).astype(int)

#Debugging: Confirm Diagnosis transformation is successful
        print("Unique values in 'Diagnosis' after preprocessing:", df['Diagnosis'].unique())
        print("Data type of 'Diagnosis':", df['Diagnosis'].dtype)

#Encode categorical variables and scale numerical features to prepare data for modelling.
#Categorical variables are converted to numerical format using Label Encoding, and
numerical features are standardised using StandardScaler.
#Parameters:df (DataFrame): The dataset after cleaning. Returns:DataFrame: The dataset
with encoded categorical variables and scaled numerical features.
```

```
    le = LabelEncoder()#Defining number for the label - will display numerically
    categorical_cols = df.select_dtypes(include=['object']).columns
    for col in categorical_cols:
        df[col] = le.fit_transform(df[col])

#Scale numerical features
    scaler = StandardScaler()
    numerical_cols = df.select_dtypes(include=['float64', 'int64']).columns
    df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

    return df

#Train Model - Train a machine learning model using Random Forest to predict asthma
diagnosis.
#The dataset is split into training and test sets, and the model is evaluated using
#Accuracy, classification report, and a confusion matrix. Parameters: df (DataFrame): The
preprocessed dataset.

#Train Model
def train_model(df):
    X = df.drop(columns=['Diagnosis'])  #Features
    y = df['Diagnosis']  #Target variable

#Debugging: Ensure y is properly formatted
    print("\nDebugging: Target variable (y) properties")
    print("Unique values in y:", y.unique())
    print("Data type of y:", y.dtype)

#Explicitly convert y to a NumPy array of integer type
    y = y.values.astype(int)  # Ensure y is an integer array
    print("\nAfter explicit conversion:")
    print("Unique values in y:", np.unique(y))
    print("Data type of y:", type(y), "with dtype:", y.dtype)

#Split data into training/testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialise and train the Random Forest Classifier
    model = RandomForestClassifier(n_estimators=100, random_state=42)
    model.fit(X_train, y_train)

#Make predictions
    y_pred = model.predict(X_test)

#Evaluate the model
    print("\nModel Performance:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Classification Report:")
    print(classification_report(y_test, y_pred, zero_division=0))
```

```
#Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title("Confusion Matrix")
    plt.show()

def train_DT_model(df):
    X = df.drop(columns=['Diagnosis'])  #Features
    y = df['Diagnosis']  #Target variable

#Debugging: Ensure y is properly formatted
    print("\nDebugging: Target variable (y) properties")
    print("Unique values in y:", y.unique())
    print("Data type of y:", y.dtype)

#Explicitly convert y to a NumPy array of integer type
    y = y.values.astype(int)  # Ensure y is an integer array
    print("\nAfter explicit conversion:")
    print("Unique values in y:", np.unique(y))
    print("Data type of y:", type(y), "with dtype:", y.dtype)

#Split data into training/testing sets
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Initialise and train the Random Forest Classifier
    model = DecisionTreeClassifier(class_weight='balanced', max_depth=4, random_state=42)
    model.fit(X_train, y_train)

#Make predictions
    y_pred = model.predict(X_test)

#Evaluate the model
    print("\nModel Performance:")
    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Classification Report:")
    print(classification_report(y_test, y_pred, zero_division=0))

#Confusion Matrix
    cm = confusion_matrix(y_test, y_pred)
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
    plt.xlabel("Predicted")
    plt.ylabel("Actual")
    plt.title("Confusion Matrix")
    plt.show()

#Plot the Decision Tree
    plt.figure(figsize=(20, 10))  #Adjust depending on the tree's size
```

```
    plot_tree(
        model,
        # feature_names=feature_names,
        # class_names=class_names,
        filled=True,
        rounded=True
        )
    plt.title("Decision Tree Visualization")
    plt.show()

    return model

#Training a DecisionTreeClassifier and visualising the resulting decision tree.
def train_and_plot_decision_tree(X, y, feature_names, class_names):

#Ensure the target variable is properly formatted
    print("\nDebugging: Target variable (y)")
    print("Unique values in y:", np.unique(y))
    print("Data type of y:", type(y), "with dtype:", y.dtype)

#Explicitly convert y to integer type if necessary
    if not np.issubdtype(y.dtype, np.integer):
        print("Warning: Target variable is not discrete! Converting to integer...")
        y = y.astype(int)

#File path defined
df = load_data(file_path)

#Cleaning the data
df_Clean = clean_data(df)

#Exploring the data that is clean
explore_data(df_Clean)

#Preprocess data in the clean 0 and 1 to represent diagnosis
df_Pre = preprocess_data(df_Clean)

df_Pre.head()#Shows the table and values

train_model(df_Pre) #Random Forest

train_DT_model(df_Pre) #Decision tree

#Main Execution
if __name__ == "__main__":
    file_path = "Documents/Transformed_Asthma_Dataset (1).csv"
    df = load_data(file_path)
    df = clean_data(df)
    explore_data(df)
    df = preprocess_data(df)
```

```
    train_model(df)

#Prepare features and target
    X = df.drop(columns=['Diagnosis'])  # Features
    y = df['Diagnosis']  # Target variable

#Double-check the target variable
    print("Target (Diagnosis) distribution:\n", y.value_counts())
    print("Target data type:", y.dtype)

#Feature and class names
    feature_names = X.columns.tolist()
    class_names = ['No', 'Yes']  # Adapt if necessary

#Train and visualize the decision tree
    train_and_plot_decision_tree(X, y, feature_names, class_names)

#Ethical Considerations and Sustainability
# - Ensure patient data is anonymised and used ethically.
# - Avoid biases in the dataset, such as underrepresenting certain ethnicities or genders.
# - Model fairness should be evaluated to prevent discrimination in asthma diagnosis.
# - Consider the environmental impact of computational resources used in model training.

#Habeeb's Code
# Define combinations to plot
combinations = [
    # Respiratory function by diagnosis
    {'numeric': 'LungFunctionFEV1', 'categorical': 'Diagnosis'},
    {'numeric': 'LungFunctionFVC', 'categorical': 'Diagnosis'},

    # Physical characteristics by diagnosis
    {'numeric': 'BMI', 'categorical': 'Diagnosis'},
    {'numeric': 'Age', 'categorical': 'Diagnosis'},
    {'numeric': 'PhysicalActivity', 'categorical': 'Diagnosis'},

    # Environmental exposures by diagnosis
    {'numeric': 'PollutionExposure', 'categorical': 'Diagnosis'},
    {'numeric': 'PollenExposure', 'categorical': 'Diagnosis'},
    {'numeric': 'DustExposure', 'categorical': 'Diagnosis'},

    # Lifestyle factors by diagnosis
    {'numeric': 'DietQuality', 'categorical': 'Diagnosis'},
    {'numeric': 'SleepQuality', 'categorical': 'Diagnosis'},

    # Respiratory function by symptoms
    {'numeric': 'LungFunctionFEV1', 'categorical': 'Wheezing'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'ShortnessOfBreath'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'ChestTightness'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'NighttimeSymptoms'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'ExerciseInduced'},
```

```
    # Respiratory function by risk factors
    {'numeric': 'LungFunctionFEV1', 'categorical': 'Smoking'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'BMI_Type'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'FamilyHistoryAsthma'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'HistoryOfAllergies'},

    # BMI relationships
    {'numeric': 'BMI', 'categorical': 'Gender'},
    {'numeric': 'BMI', 'categorical': 'Ethnicity'},
    {'numeric': 'BMI', 'categorical': 'Smoking'},

    # Age relationships
    {'numeric': 'Age', 'categorical': 'Smoking'},
    {'numeric': 'Age', 'categorical': 'HistoryOfAllergies'},
    {'numeric': 'Age', 'categorical': 'ExerciseInduced'},

    # Physical activity relationships
    {'numeric': 'PhysicalActivity', 'categorical': 'Gender'},
    {'numeric': 'PhysicalActivity', 'categorical': 'BMI_Type'},
    {'numeric': 'PhysicalActivity', 'categorical': 'ExerciseInduced'},

    # Environmental exposures by symptoms
    {'numeric': 'PollenExposure', 'categorical': 'HayFever'},
    {'numeric': 'DustExposure', 'categorical': 'Wheezing'},
    {'numeric': 'PollutionExposure', 'categorical': 'ShortnessOfBreath'}
]

# Create a figure for each combination
for combo in combinations:
    numeric_var = combo['numeric']
    categorical_var = combo['categorical']

    plt.figure(figsize=(10, 6))

    # Create the boxplot
    sns.boxplot(x=categorical_var, y=numeric_var, data=df)

    # Add title and labels
    plt.title(f'Distribution of {numeric_var} by {categorical_var}', fontsize=16)
    plt.xlabel(categorical_var, fontsize=14)
    plt.ylabel(numeric_var, fontsize=14)

    # Add a grid for easier reading
    plt.grid(axis='y', linestyle='--', alpha=0.7)

    # Save the figure
    plt.savefig(f'boxplot_{numeric_var}_by_{categorical_var}.png', dpi=300,
bbox_inches='tight')
```

```
    # Show the plot
    plt.show()

#Naren's Code
# Set the style for plots
plt.style.use('seaborn-v0_8-whitegrid')
sns.set(font_scale=1.2)

# Display settings for better visualization
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)

#Loads the data and explores its structure

# Display basic information
print("Dataset shape:", df.shape)
print("\nFirst few rows:")
display(df.head())

# Check for missing values
print("\nMissing values per column:")
print(df.isnull().sum())

# Display summary statistics
print("\nSummary statistics:")
display(df.describe())

#Data Processing - preprocesses the data converting into categorical variables into more
meaningful representations

# Create a copy of the dataframe for analysis
df_analysis = df.copy()

# Convert categorical variables
# Assuming Gender: 0=Male, 1=Female
df_analysis['Gender'] = df_analysis['Gender'].map({0: 'Male', 1: 'Female'})

# Assuming Diagnosis: 0=No Asthma, 1=Asthma
df_analysis['Diagnosis'] = df_analysis['Diagnosis'].map({0: 'No Asthma', 1: 'Asthma'})

# Display the updated dataframe
display(df_analysis.head())

#Correlation Analysis - examines the correlation between numerical variables in dataset.

# Create a correlation matrix for numerical variables
numeric_df = df.select_dtypes(include=[np.number])
correlation_matrix = numeric_df.corr()

# Plot correlation heatmap
```

```
plt.figure(figsize=(14, 12))
sns.heatmap(correlation_matrix, annot=False, cmap='coolwarm', linewidths=0.5)
plt.title('Correlation Matrix of Numerical Variables')
plt.tight_layout()
plt.show()

# Display the correlation with Diagnosis
diagnosis_corr = correlation_matrix['Diagnosis'].sort_values(ascending=False)
print("\nCorrelation with Diagnosis:")
print(diagnosis_corr)

import os
import matplotlib.pyplot as plt

# Ensure 'plots' directory exists to begin with in area
os.makedirs('plots', exist_ok=True)

# Defining the symptom columns to group by diagnosis
symptom_cols = ['Wheezing', 'ShortnessOfBreath', 'ChestTightness', 'Coughing',
            'NighttimeSymptoms', 'ExerciseInduced']

# Filter for only Asthma patients only removing the non asthma patients from data displayed
in graph
asthma_df = df[df['Diagnosis'] == 1]

# Calculate average symptom rates for Asthma group only
asthma_rates = asthma_df[symptom_cols].mean().sort_values(ascending=False)

# Plotting with legend
plt.figure(figsize=(10, 6))
asthma_rates.plot(kind='bar', color='teal')
plt.title('Symptom Rates by Diagnosis')
plt.ylabel('Rate')
plt.xlabel('Symptoms')
plt.xticks(rotation=45)
plt.legend(title='Diagnosis Group - Asthma')  # This adds the key to the bar chart
plt.tight_layout()

# Save and show
plt.savefig('plots/symptoms_by_diagnosis.png')
plt.show()

# Analyze relationship between symptoms and diagnosis with p-values
symptom_cols = ['Wheezing', 'ShortnessOfBreath', 'ChestTightness', 'Coughing',
            'NighttimeSymptoms', 'ExerciseInduced']

# Calculate chi-square p-values for each symptom
symptom_pvalues = {}
for symptom in symptom_cols:
    contingency_table = pd.crosstab(df[symptom], df['Diagnosis'])
```

```
    chi2, p, dof, expected = stats.chi2_contingency(contingency_table)
    symptom_pvalues[symptom] = p

# Display p-values
print("Chi-square test p-values for symptoms vs diagnosis:")
for symptom, p_value in symptom_pvalues.items():
    significance = "Significant" if p_value < 0.05 else "Not significant"
    print(f"{symptom}: p-value = {p_value:.4f} ({significance})")

#Age & BMI Analysis - examining the relationship between age, BMI and the diagnosis

#Creating Box plot from data of age and having asthma and not having asthma
sns.boxplot(data=df_analysis, x='Diagnosis', y='Age', palette='Set2')
# Customize axes
plt.title('Age Distribution by Diagnosis')
plt.xlabel('Diagnosis')
plt.ylabel('Age')
plt.xticks([0, 1], ['No Asthma', 'Asthma'])

#Creating another boxplot with BMI and the diagnosis
sns.boxplot(data=df_analysis, x='Diagnosis', y='BMI', palette='Set3')
# Title and labels
plt.title('BMI Distribution by Diagnosis')
plt.xlabel('Diagnosis')
plt.ylabel('BMI')
plt.xticks([0, 1], ['No Asthma', 'Asthma'])

# T-test for BMI between diagnosis groups
asthma_bmi = df[df['Diagnosis'] == 1]['BMI']
no_asthma_bmi = df[df['Diagnosis'] == 0]['BMI']
t_stat, p_value = stats.ttest_ind(asthma_bmi, no_asthma_bmi, equal_var=False)
print(f"T-test for BMI between Asthma and No Asthma groups:")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Significance: {'Significant' if p_value < 0.05 else 'Not significant'}")

# T-test for Age between diagnosis groups
asthma_age = df[df['Diagnosis'] == 1]['Age']
no_asthma_age = df[df['Diagnosis'] == 0]['Age']
t_stat, p_value = stats.ttest_ind(asthma_age, no_asthma_age, equal_var=False)
print(f"\nT-test for Age between Asthma and No Asthma groups:")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Significance: {'Significant' if p_value < 0.05 else 'Not significant'}")

# Lung Function Analysis - analysing the lung function metrics *FEV1 and the FVC* by
their diagnosis

# Set up the overall figure with 2 subplots side-by-side
fig, axes = plt.subplots(1, 2, figsize=(14, 6), sharey=False)
```

```
# First plot: FEV1 by Diagnosis
sns.boxplot(data=df_analysis, x='Diagnosis', y='LungFunctionFEV1', palette='coolwarm',
ax=axes[0])
axes[0].set_title('FEV1 by Diagnosis')
axes[0].set_xlabel('Diagnosis')
axes[0].set_ylabel('FEV1')
plt.xticks([0, 1], ['No Asthma', 'Asthma'])

# Second plot: FVC by Diagnosis
sns.boxplot(data=df_analysis, x='Diagnosis', y='LungFunctionFVC', palette='coolwarm',
ax=axes[1])
axes[1].set_title('FVC by Diagnosis')
axes[1].set_xlabel('Diagnosis')
axes[1].set_ylabel('FVC')

# Adjust layout and save
plt.tight_layout()
plt.savefig('plots/fev1_fvc_boxplots.png')
plt.show()


# T-test for lung function between diagnosis groups
asthma_fev1 = df[df['Diagnosis'] == 1]['LungFunctionFEV1']
no_asthma_fev1 = df[df['Diagnosis'] == 0]['LungFunctionFEV1']
t_stat, p_value = stats.ttest_ind(asthma_fev1, no_asthma_fev1, equal_var=False)
print(f"T-test for FEV1 between Asthma and No Asthma groups:")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Significance: {'Significant' if p_value < 0.05 else 'Not significant'}")

asthma_fvc = df[df['Diagnosis'] == 1]['LungFunctionFVC']
no_asthma_fvc = df[df['Diagnosis'] == 0]['LungFunctionFVC']
t_stat, p_value = stats.ttest_ind(asthma_fvc, no_asthma_fvc, equal_var=False)
print(f"\nT-test for FVC between Asthma and No Asthma groups:")
print(f"t-statistic: {t_stat:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Significance: {'Significant' if p_value < 0.05 else 'Not significant'}")

# Defining the risk factors
risk_factors = ['Smoking', 'PhysicalActivity', 'PollutionExposure',
            'PollenExposure', 'DustExposure', 'PetAllergy',
            'FamilyHistoryAsthma', 'HistoryOfAllergies', 'Eczema', 'HayFever']

# Calculating the prevalence for asthma and no asthma groups
prevalence = {}
for factor in risk_factors:
    asthma_prevalence = df[df['Diagnosis'] == 1][factor].mean()  # Mean prevalence for
asthma group being positive for having Asthma
```

```
    no_asthma_prevalence = df[df['Diagnosis'] == 0][factor].mean()  # Mean prevalence for no
asthma group who do not have Asthma diagnosis
    prevalence[factor] = {'Asthma': asthma_prevalence, 'No Asthma': no_asthma_prevalence}

# Convert prevalence dictionary to DataFrame
prevalence_df = pd.DataFrame(prevalence).T  # Transposing to get factors as rows

# Plot the bar chart from set parameters
prevalence_df.plot(kind='bar', figsize=(12, 6))
plt.title('Risk Factors vs Prevalence in Asthma and No Asthma Groups')
plt.xlabel('Risk Factors')
plt.ylabel('Prevalence')
plt.xticks(rotation=45)
plt.legend(title='Group')
plt.tight_layout()
plt.show()
# Risk factors analysis with p-values
risk_factors = ['Smoking', 'PhysicalActivity', 'PollutionExposure',
                'PollenExposure', 'DustExposure', 'PetAllergy',
                'FamilyHistoryAsthma', 'HistoryOfAllergies', 'Eczema', 'HayFever']

# Calculate p-values for each risk factor
risk_factor_pvalues = {}
for factor in risk_factors:
    if df[factor].nunique() <= 2:
        # For binary factors, use chi-square test
        contingency_table = pd.crosstab(df[factor], df['Diagnosis'])
        chi2, p, dof, expected = stats.chi2_contingency(contingency_table)
        test_type = "Chi-square"
    else:
        # For continuous factors, use t-test
        asthma_val = df[df['Diagnosis'] == 1][factor]
        no_asthma_val = df[df['Diagnosis'] == 0][factor]
        t_stat, p = stats.ttest_ind(asthma_val, no_asthma_val, equal_var=False)
        test_type = "T-test"

    risk_factor_pvalues[factor] = (p, test_type)

# Display p-values
print("Statistical test p-values for risk factors vs diagnosis:")
for factor, (p_value, test_type) in risk_factor_pvalues.items():
    significance = "Significant" if p_value < 0.05 else "Not significant"
    print(f"{factor}: p-value = {p_value:.4f} ({test_type}, {significance})")

# Prepare data for logistic regression
X = df[['Age', 'BMI', 'Smoking', 'PhysicalActivity', 'PollutionExposure',
        'PollenExposure', 'DustExposure', 'PetAllergy', 'FamilyHistoryAsthma',
        'HistoryOfAllergies', 'Eczema', 'HayFever', 'LungFunctionFEV1',
        'LungFunctionFVC', 'Wheezing', 'ShortnessOfBreath', 'ChestTightness',
        'Coughing', 'NighttimeSymptoms', 'ExerciseInduced']]
```

```
y = df['Diagnosis']

# Fit statsmodels logistic regression for p-values
X_sm = sm.add_constant(X)
logit_model = sm.Logit(y, X_sm)
result = logit_model.fit()

# Display summary
print("Logistic Regression Summary:")
print(result.summary())

# Extract p-values and create a summary table
pvalues = result.pvalues[1:]
coefs = result.params[1:]
odds_ratios = np.exp(coefs)

# Create a summary dataframe
summary_df = pd.DataFrame({
    'Contributing_Factors': X.columns,
    'Coefficient': coefs,
    'Odds Ratio': odds_ratios,
    'p-value': pvalues,
    'Significant': pvalues < 0.05
})

# Sort by p-value
summary_df = summary_df.sort_values('p-value')

# Display the summary
display(summary_df)

# Use the sorted summary DataFrame from your code
sorted_summary_df = summary_df.sort_values('Coefficient', ascending=False)

# Create a horizontal bar chart
plt.figure(figsize=(14, 8))
plt.barh(sorted_summary_df['Contributing_Factors'], sorted_summary_df['Coefficient'],
color='skyblue')
plt.axvline(0, color='red', linestyle='--', linewidth=1)  # Add a vertical reference line at 0
plt.title('Logistic Regression Coefficients for Asthma Diagnosis')
plt.xlabel('Coefficient Value')
plt.ylabel('Contributing Factors')
plt.tight_layout()
plt.show()

#Aprem's Code - Unsupervised learning
print(df.head())  # View first few rows
print(df.info())  # Check data types
print(df.describe())  # Get summary statistics
```

```
# Since machine learning models work best with numerical data, convert categorical
variables into numerical format.
# Convert Yes/No columns to 1/0
df.replace({"Yes": 1, "No": 0}, inplace=True)
print(df.head())

# For true categorical features without inherent order use One-Hot Encoding.
categorical_cols = ["Gender", "Ethnicity"]

df_encoded = pd.get_dummies(df, columns=categorical_cols, drop_first=False)
print(df_encoded.head())

# K-Means relies on Euclidean distance, which needs numerical values. Hence convert any
boolean columns to integers
for col in df_encoded.columns:
    if df_encoded[col].dtype == bool:
        df_encoded[col] = df_encoded[col].astype(int)
print(df_encoded.head())

# Standardise the Features
#This ensures all features have equal weight in distance calculations.

get_ipython().system('pip install scikit-learn')

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
df_standardized = scaler.fit_transform(df_encoded)

# After standardization the dataset is a NumPy array. And NumPy arrays do not have the
head() method that is specific to Pandas DataFrames.
# To resolve this, we are converting df_standardized back to a Pandas DataFrame

df_standardized = pd.DataFrame(df_standardized, columns=df_encoded.columns)

print(df_standardized.head())

#Use the Elbow Method to Choose k (number of clusters)

import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

inertia = []
k_range = range(1, 50)

for k in k_range:
    model = KMeans(n_clusters=k, random_state=42)
    model.fit(df_standardized)
    inertia.append(model.inertia_)

plt.figure(figsize=(8, 5))
```

```
plt.plot(k_range, inertia, marker='o')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia (WCSS)')
plt.title('Elbow Method For Optimal k')
plt.grid(True)
plt.show()

# Optimal K calculated from elbow method = 14 ("elbow" point where the drop in inertia
slows)
optimal_k = 14

kmeans = KMeans(n_clusters=optimal_k, random_state=42)
clusters = kmeans.fit_predict(df_standardized)

# Add cluster labels back to the original dataframe
df_encoded["Cluster"] = clusters
print(df_encoded.head())

# Analyze the Clusters
# View basic cluster distribution
print(df_encoded["Cluster"].value_counts())

# Check cluster-wise means
cluster_means = df_encoded.groupby("Cluster")["Diagnosis"].mean()
print(cluster_means)

# List of feature columns you used for clustering (excluding "Cluster")
features = [col for col in df_encoded.columns if col != "Cluster"]

# Calculate the mean of each feature per cluster

cluster_summary = df_encoded.groupby("Cluster")[features].mean().round(2)
print(cluster_summary)

# Export as Excel File
get_ipython().system('pip install openpyxl')
cluster_summary.to_excel("cluster_summary.xlsx", index=True)

import seaborn as sns

plt.figure(figsize=(20, 10))
sns.heatmap(cluster_summary, annot=True, cmap="coolwarm")
plt.title("Cluster Feature Means")
plt.tight_layout()

# Save the heatmap as a PNG image
plt.savefig("cluster_summary_heatmap.png", dpi=300)
plt.show()

import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns
# Read the dataset
def load_data(file_path):
    df = pd.read_csv(file_path)
    return df
# Define combinations to plot
    combinations = [
# Respiratory function by diagnosis
    {'numeric': 'LungFunctionFEV1', 'categorical': 'Diagnosis'},
    {'numeric': 'LungFunctionFVC', 'categorical': 'Diagnosis'},
# Physical characteristics by diagnosis
    {'numeric': 'BMI', 'categorical': 'Diagnosis'},
    {'numeric': 'Age', 'categorical': 'Diagnosis'},
    {'numeric': 'PhysicalActivity', 'categorical': 'Diagnosis'},
# Environmental exposures by diagnosis
    {'numeric': 'PollutionExposure', 'categorical': 'Diagnosis'},
    {'numeric': 'PollenExposure', 'categorical': 'Diagnosis'},
    {'numeric': 'DustExposure', 'categorical': 'Diagnosis'},
# Lifestyle factors by diagnosis
    {'numeric': 'DietQuality', 'categorical': 'Diagnosis'},
    {'numeric': 'SleepQuality', 'categorical': 'Diagnosis'},
# Respiratory function by symptoms
    {'numeric': 'LungFunctionFEV1', 'categorical': 'Wheezing'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'ShortnessOfBreath'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'ChestTightness'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'NighttimeSymptoms'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'ExerciseInduced'},
# Respiratory function by risk factors
    {'numeric': 'LungFunctionFEV1', 'categorical': 'Smoking'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'BMI_Type'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'FamilyHistoryAsthma'},
    {'numeric': 'LungFunctionFEV1', 'categorical': 'HistoryOfAllergies'},
# BMI relationships
    {'numeric': 'BMI', 'categorical': 'Gender'},
    {'numeric': 'BMI', 'categorical': 'Ethnicity'},
    {'numeric': 'BMI', 'categorical': 'Smoking'},
# Age relationships
    {'numeric': 'Age', 'categorical': 'Smoking'},
    {'numeric': 'Age', 'categorical': 'HistoryOfAllergies'},
    {'numeric': 'Age', 'categorical': 'ExerciseInduced'},
# Physical activity relationships
    {'numeric': 'PhysicalActivity', 'categorical': 'Gender'},
    {'numeric': 'PhysicalActivity', 'categorical': 'BMI_Type'},
    {'numeric': 'PhysicalActivity', 'categorical': 'ExerciseInduced'},
# Environmental exposures by symptoms
    {'numeric': 'PollenExposure', 'categorical': 'HayFever'},
    {'numeric': 'DustExposure', 'categorical': 'Wheezing'},
    {'numeric': 'PollutionExposure', 'categorical': 'ShortnessOfBreath'}
    ]
```

```
# Create a figure for each combination
    for combo in combinations:
        numeric_var = combo['numeric']
        categorical_var = combo['categorical']
        plt.figure(figsize=(10, 6))

# Create the boxplot
sns.boxplot(x=categorical_var, y=numeric_var, data=df)
# Add title and labels
plt.title(f'Distribution of {numeric_var} by {categorical_var}', fontsize=16)
plt.xlabel(categorical_var, fontsize=14)
plt.ylabel(numeric_var, fontsize=14)
# Add a grid for easier reading
plt.grid(axis='y', linestyle='--', alpha=0.7)
# Save the figure
plt.savefig(f'boxplot_{numeric_var}_by_{categorical_var}.png', dpi=300,
bbox_inches='tight')
# Show the plot
plt.show()

#Aprem Graphical Representations
print(df.head())  # View first few rows
print(df.info())  # Check data types
print(df.describe())  # Get summary statistics

category_col = "Diagnosis"

# possible cause factors of asthma (having numerical values)
numerical_cols = ["BMI","PhysicalActivity", "DietQuality","SleepQuality",
"PollutionExposure", "PollenExposure", "DustExposure"]

df_melted = df.melt(id_vars=[category_col],
                    value_vars=numerical_cols,
                    var_name="Metric",
                    value_name="Value")


plt.figure(figsize=(30, 18))  # Increased figure size
# create boxplot of cause factors by diagnosis result
sns.boxplot(x=category_col, y="Value", hue="Metric", data=df_melted, palette="Set2")

plt.title("Comparison of possible contributors(having numerical values) by diagnosis result",
fontsize=16)
plt.xlabel("Diagnosis (Yes/No)", fontsize=14)
plt.ylabel("Value", fontsize=14)

# Set y-axis scale with a step of 1
y_min = int(df_melted["Value"].min())  # Convert to integer to avoid float-based ticks
y_max = int(df_melted["Value"].max()) + 1
plt.ylim(y_min, y_max)
```

```
plt.yticks(range(y_min, y_max + 1, 1))  # Step of 1

plt.legend(title="Metric", bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0., fontsize=20,
markerscale=3)
plt.grid(True)
plt.show()

plt.savefig("boxplot.png", dpi=300, bbox_inches="tight")

plt.figure(figsize=(30, 18))  # Increased figure size

# Create a point plot instead of a box plot
sns.pointplot(x=category_col, y="Value", hue="Metric", data=df_melted, palette="Set2",
            dodge=True, markers="o", linestyles="--",
            markersize=10, linewidth=3)  # Using markersize and linewidth instead of scale

plt.title("Comparison of possible contributors (having numerical values) by diagnosis result",
fontsize=16)
plt.xlabel("Diagnosis (Yes/No)", fontsize=14)
plt.ylabel("Value", fontsize=14)

# Set y-axis scale with a step of 1
y_min = int(df_melted["Value"].min())  # Convert to integer to avoid float-based ticks
y_max = int(df_melted["Value"].max()) + 1
plt.ylim(y_min, y_max)
plt.yticks(range(y_min, y_max + 1, 1))  # Step of 1

# Adjust legend position and font size
plt.legend(title="Metric", bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0., fontsize=20,
markerscale=3)

plt.grid(True)
plt.show()

numerical_cols = ["LungFunctionFEV1", "LungFunctionFVC"]

df_melted = df.melt(id_vars=[category_col],
                value_vars=numerical_cols,
                var_name="Metric",
                value_name="Value")


plt.figure(figsize=(30, 18))  # Increased figure size
sns.boxplot(x=category_col, y="Value", hue="Metric", data=df_melted, palette="Set2")

plt.title("Comparison of lung health measurements by diagnosis result", fontsize=24)
plt.xlabel("Diagnosis (Yes/No)", fontsize=22)
plt.ylabel("Value", fontsize=22)

# Set y-axis scale with a step of 1
```

```
y_min = int(df_melted["Value"].min())  # Convert to integer to avoid float-based ticks
y_max = int(df_melted["Value"].max()) + 1
plt.ylim(y_min, y_max)
plt.yticks(np.arange(np.floor(y_min), np.ceil(y_max), 0.5))

plt.legend(title="Metric", bbox_to_anchor=(1.05, 1), loc=2, borderaxespad=0., fontsize=24,
markerscale=4)
# Increase size of x and y axis tick labels
plt.xticks(fontsize=20)  # Larger x-axis ticks (Yes/No)
plt.yticks(fontsize=20)  # Larger y-axis ticks (numbers)
plt.grid(True)
plt.show()

# Convert categorical Yes/No values into numerical 1/0. This is done for createing a heatmap,
since heatmap can only be applied on numerical columns
df_binary = df.replace({"Yes": 1, "No": 0})

# Display the transformed dataset
print(df_binary.head())

# Selecting the relevant columns for analysis
print(df.columns)

# Exclude columns which are not symptoms
X = df.drop(columns= ["Age", "Gender", "Ethnicity", "BMI",
        "BMI_Type", "Smoking", "PhysicalActivity", "DietQuality", "SleepQuality",
"PollutionExposure", "PollenExposure",
        "DustExposure", "PetAllergy", "FamilyHistoryAsthma", "HistoryOfAllergies",
"LungFunctionFEV1", "LungFunctionFVC", "ExerciseInduced"])

plt.figure(figsize=(10, 6))  # Adjust figure size

# Generate heatmap of correlations between possible symptoms and diagnosis
sns.heatmap(X.corr(), annot=True, cmap="coolwarm", cbar=True, linewidths=0.5)

plt.title("Heatmap of Correlations Between Symptoms and Diagnosis")
plt.show()

X = df.drop(columns=
["Age","Gender","Ethnicity","BMI","BMI_Type","Smoking","PhysicalActivity","DietQualit
y","SleepQuality","PollutionExposure","PollenExposure","DustExposure","PetAllergy","Fa
milyHistoryAsthma","HistoryOfAllergies","LungFunctionFEV1","LungFunctionFVC","Exer
ciseInduced"])  # Exclude non-numeric columns

plt.figure(figsize=(10, 6))  # Adjust figure size
sns.heatmap(X.groupby("Diagnosis").mean(), annot=True, cmap="coolwarm", cbar=True)

plt.title("Heatmap of Average Yes/No Occurrences of symptoms by Diagnosis")
plt.show()
```

```
# In below cells we are finding the mean and standard deviation of possible cause factors(
having numerical values) in order to estimate a benchmark value for that particular cause
factor.
# By using this benchmark value as a criteria the column values can be converted to either 0
or 1 which is suitable for heatmap generation.

column_name = "DietQuality"

mean_value = df[column_name].mean()  # DietQuality mean
std_dev_value = df[column_name].std()  # DietQuality standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")

column_name = "SleepQuality"

mean_value = df[column_name].mean()  # SleepQuality mean
std_dev_value = df[column_name].std()  # SleepQuality standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")

column_name = "PollutionExposure"

mean_value = df[column_name].mean()  # PollutionExposure mean
std_dev_value = df[column_name].std()  # PollutionExposure standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")

column_name = "PollenExposure"

mean_value = df[column_name].mean()  # PollenExposure mean
std_dev_value = df[column_name].std()  # PollenExposure standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")

column_name = "DustExposure"

mean_value = df[column_name].mean()  # DustExposure mean
std_dev_value = df[column_name].std()  # DustExposure standard deviation

print(f"Mean of {column_name}: {mean_value}")
print(f"Standard Deviation of {column_name}: {std_dev_value}")

# The column values are converted to either 0 or 1 based on the respective benchmark values.
```

```
# Change it to 1 if less than the benchmark value (factors which should be flagged if less than the benchmark value). Else change it to zero.

df["PhysicalActivity"] = np.where(df["PhysicalActivity"] < 3.5, 1, 0)

df["DietQuality"] = np.where(df["DietQuality"] < 3.6, 1, 0)

df["SleepQuality"] = np.where(df["SleepQuality"] < 5, 1, 0)

# Change it to 1 if greater than the benchmark value (factors which should be flagged if greater than the benchmark value)

df["PollutionExposure"] = np.where(df["PollutionExposure"] > 7.93, 1, 0)

df["PollenExposure"] = np.where(df["PollenExposure"] > 8, 1, 0)

df["DustExposure"] = np.where(df["DustExposure"] > 7.83, 1, 0)

print(df.head())

# The column values are converted to either 0 or 1 based on the benchmark values of BMI.

df["BMI"] = np.where((df["BMI"] < 16) | (df["BMI"] > 30), 1, 0)
print(df.head(20))

# Convert categorical Yes/No values into numerical 1/0. This is done for createing a heatmap, since heatmap can only be applied on numerical columns
df_binary = df.replace({"Yes": 1, "No": 0})

# Display the transformed dataset
print(df_binary.head())

# Include only possible cause factor columns
cause_cols = ["Diagnosis", "BMI", "Smoking",
"PhysicalActivity","DietQuality","SleepQuality","PollutionExposure","PollenExposure","DustExposure","PetAllergy","FamilyHistoryAsthma","HistoryOfAllergies"]
df_cause = df_binary[cause_cols]

plt.figure(figsize=(10, 6))  # Adjust figure size
sns.heatmap(df_cause.groupby("Diagnosis").mean(), annot=True, cmap="coolwarm", cbar=True)

plt.title("Heatmap of Average Yes/No Occurrences of causes by Diagnosis")
plt.show()

# Compute the FEV1/FVC Ratio
df["FEV1_FVC_Ratio"] = df["LungFunctionFEV1"] / df["LungFunctionFVC"]

# Apply Asthma Criteria
# Asthma is likely if:
```

```
# FEV1 < 7 (indicative of airflow limitation)
# FEV1/FVC Ratio < 0.7 (suggesting obstruction)

df["Possible_Asthma"] = (df["LungFunctionFEV1"] < 7) & (df["FEV1_FVC_Ratio"] < 0.7)

print(df.head())

# Convert True and False values of Possible_Asthma column into numerical 1/0.
df_binary = df.replace({True: 1, False: 0}).replace({"Yes": 1, "No": 0})
print(df_binary.head())

# Create Heatmap of asthma detection based on LungFunctionFEV1 and LungFunctionFVC.
detection_cols = ["Diagnosis", "Possible_Asthma"]
df_detection = df_binary[detection_cols]

plt.figure(figsize=(10, 6))  # Adjust figure size
sns.heatmap(df_detection.groupby("Diagnosis").mean(), annot=True, cmap="coolwarm",
cbar=True)

plt.title("Heatmap of asthma detection based on LungFunctionFEV1 and
LungFunctionFVC")
plt.show()
```